ĆWICZENIA DO INSTRUKCJI 4	Programowanie obiektowe (Java)
	Temat: **Odśmiecanie pamięci, referencje, pakiety, adnotacje**
	Przygotował: **mgr inż. Michał Młodawski**

CEL ZAJĘĆ

Po wykonaniu ćwiczeń powinieneś:

- utworzyć słabą referencję do obiektu i sprawdzić, kiedy garbage collector może ten obiekt usunąć,
- wyjaśnić, dlaczego `System.gc()` jest tylko prośbą do maszyny wirtualnej,
- umieścić klasę w pakiecie zgodnym ze strukturą katalogów projektu Maven,
- zbudować aplikację wiersza poleceń z adnotacjami `@CommandLine.Command` i `@CommandLine.Option` biblioteki Picocli.

Ćwiczenia sprawdzają materiał z instrukcji laboratoryjnej 4. Zadanie 1 wykonasz w jednym pliku, bez Mavena. Zadanie 2 wymaga projektu Maven z biblioteką Picocli (bez Spring Boot), więc potrzebny jest dostęp do repozytorium Maven Central.

Tabela 1. Zadania w tym zestawie

Zadanie	Temat
1	Słaba referencja pod lupą
2	Kalkulator ocen w wierszu poleceń

Zadanie 1. Słaba referencja pod lupą

ZADANIE

Napisz program `SlabaReferencja`, który pokazuje, że obiekt dostępny przez słabą referencję przetrwa odśmiecanie tylko tak długo, jak długo prowadzi do niego także silna referencja. Uruchom program dwa razy: normalnie i z opcją, która wyłącza `System.gc()`.

Wymagania:

1. Utwórz plik `SlabaReferencja.java` z klasą publiczną `SlabaReferencja` i metodą `main`.
2. Utwórz obiekt napisu poleceniem `new String("Raport z laboratorium")` i zapisz go w zmiennej `raport`.
3. Utwórz słabą referencję `slaba` typu `WeakReference<String>` do tego obiektu.
4. Poproś o odśmiecanie (`System.gc()`) i wypisz wynik `slaba.get()` z opisem `1. Silna referencja istnieje: `.
5. Przypisz zmiennej `raport` wartość `null`, ponownie poproś o odśmiecanie i wypisz wynik `slaba.get()` z opisem `2. Po usunięciu silnej referencji: `.
6. Uruchom program poleceniem `java SlabaReferencja`, a potem poleceniem `java -XX:+DisableExplicitGC SlabaReferencja` i porównaj wyniki.

Oczekiwany wynik (pierwsze uruchomienie) Tekst

```
1. Silna referencja istnieje: Raport z laboratorium
2. Po usunięciu silnej referencji: null
```

W drugim uruchomieniu drugi wiersz się zmieni. Zanim uruchomisz program, zapisz, czego się spodziewasz, i sprawdź, czy miałeś rację. Wynik drugiego wiersza w pierwszym uruchomieniu zależy od decyzji JVM. Na JDK 25 z domyślnymi ustawieniami powinieneś zobaczyć `null`, ale specyfikacja tego nie gwarantuje.

> **UWAGA**
>
> Nie twórz napisu literałem, czyli zapisem `String raport = "Raport z laboratorium";`. Instrukcja wyjaśnia, dlaczego takie obiekty nigdy nie są usuwane. Uważaj też, żeby nie zapisać wyniku `slaba.get()` w dodatkowej zmiennej przed krokiem 5: taka zmienna jest znowu silną referencją.

Zadanie 2. Kalkulator ocen w wierszu poleceń

> **ZADANIE**
>
> Napisz aplikację wiersza poleceń `oceny`, która zamienia liczbę zdobytych punktów na ocenę według skali podanej niżej. Argumenty programu ma analizować biblioteka Picocli, a program ma zwracać kod wyjścia 0 przy sukcesie i 1 przy niepoprawnych danych.

Utwórz projekt Maven o poniższej strukturze. Plik `pom.xml` jest gotowy: poza zależnością Picocli i procesorem adnotacji `picocli-codegen`, znanymi z instrukcji, zawiera wtyczkę `maven-assembly-plugin`. Dzięki niej polecenie `mvn package` zbuduje dodatkowy plik JAR, który zawiera również bibliotekę Picocli i uruchamia się poleceniem `java -jar`.

Struktura projektu Struktura

```
oceny
├── pom.xml
└── src
    └── main
        └── java
            └── pl
                └── psk
                    └── oceny
                        └── KalkulatorOcen.java
```

pom.xml XML

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                             https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>pl.psk</groupId>
    <artifactId>oceny</artifactId>
    <version>1.0</version>

    <properties>
        <maven.compiler.release>25</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <picocli.version>4.7.7</picocli.version>
    </properties>

    <dependencies>
        <dependency>
```

```
            <groupId>info.picocli</groupId>
            <artifactId>picocli</artifactId>
            <version>${picocli.version}</version>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.16.0</version>
                <configuration>
                    <annotationProcessorPaths>
                        <path>
                            <groupId>info.picocli</groupId>
                            <artifactId>picocli-codegen</artifactId>
                            <version>${picocli.version}</version>
                        </path>
                    </annotationProcessorPaths>
                </configuration>
            </plugin>
            <!-- buduje dodatkowy plik JAR z programem i biblioteką Picocli -->
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-assembly-plugin</artifactId>
                <version>3.8.0</version>
                <configuration>
                    <descriptorRefs>
                        <descriptorRef>jar-with-dependencies</descriptorRef>
                    </descriptorRefs>
                    <archive>
                        <manifest>
                            <mainClass>pl.psk.oceny.KalkulatorOcen</mainClass>
                        </manifest>
                    </archive>
                </configuration>
                <executions>
                    <execution>
                        <phase>package</phase>
                        <goals>
                            <goal>single</goal>
                        </goals>
                    </execution>
                </executions>
            </plugin>
        </plugins>
    </build>
</project>
```

Wymagania:

1. Klasa `KalkulatorOcen` należy do pakietu `pl.psk.oceny`, implementuje interfejs `Callable<Integer>` i ma adnotację `@CommandLine.Command` z nazwą `oceny`, opisem „Zamienia liczbę punktów na ocenę”, wersją `oceny 1.0` i standardowymi opcjami pomocy.
2. Zdefiniuj trzy opcje (adnotacja `@CommandLine.Option` nad polem):

- `-p`, `--punkty`: liczba zdobytych punktów, typ `int`, opcja obowiązkowa,
- `-m`, `--maks`: maksymalna liczba punktów, typ `int`, domyślnie 100,
- `-i`, `--imie`: imię studenta, typ `String`, domyślnie „Student”.

3. Napisz metodę statyczną `double ocena(double procent)`, która zamienia wynik procentowy na ocenę według tabeli poniżej.
4. W metodzie `call()` sprawdź dane. Jeśli maksimum nie jest dodatnie albo liczba punktów wykracza poza zakres od 0 do maksimum, wypisz komunikat na standardowe wyjście błędów (`System.err`) i zwróć 1. W przeciwnym razie wypisz dwa wiersze w formacie z przykładu i zwróć 0.
5. W metodzie `main` uruchom polecenie przez `new CommandLine(...).execute(args)` i przekaż otrzymany kod do `System.exit()`.

Tabela 2. Skala ocen w zadaniu (każdy próg obowiązuje aż do następnego, np. 69,9 procent to jeszcze ocena 3,5)

Wynik procentowy	poniżej 50	od 50	od 60	od 70	od 80	od 90
Ocena	2,0	3,0	3,5	4,0	4,5	5,0

Przykładowe uruchomienia Terminal

```
> mvn package
> java -jar target/oceny-1.0-jar-with-dependencies.jar -p 78
Student: 78 z 100 pkt (78,0 procent)
Ocena: 4,0
> java -jar target/oceny-1.0-jar-with-dependencies.jar -p 45 --maks 50 -i Anna
Anna: 45 z 50 pkt (90,0 procent)
Ocena: 5,0
> java -jar target/oceny-1.0-jar-with-dependencies.jar -p 120
Liczba punktów musi być z zakresu od 0 do 100
> java -jar target/oceny-1.0-jar-with-dependencies.jar -p abc
Invalid value for option '--punkty': 'abc' is not an int
Usage: oceny [-hV] [-i=<imie>] [-m=<maks>] -p=<punkty>
...
```

Wykonaj też `--help` i `-V` oraz sprawdź kod wyjścia każdego uruchomienia (Linux i macOS: `echo $?`, PowerShell: `echo $LASTEXITCODE`). Argumenty możesz też podać w IntelliJ IDEA w polu *Program arguments*. Separator dziesiętny zależy od ustawień regionalnych systemu.

> **WSKAZÓWKA**
>
> Wróć do klasy `MyCommand` z instrukcji i do ramki o Picocli bez Springa. Obowiązkowość opcji ustawia jeden atrybut adnotacji, a wartość domyślną zapisujesz po prostu przy deklaracji pola. Zamianą tekstu `"78"` na liczbę `int` zajmuje się Picocli, więc nie potrzebujesz `Integer.parseInt()`.

Sprawdź się

Zanim oddasz rozwiązania, upewnij się, że:

- w zadaniu 1 potrafisz wyjaśnić, dlaczego pierwszy wiersz zawsze pokazuje napis, a drugi zależy od opcji `-XX:+DisableExplicitGC`,
- w zadaniu 2 deklaracja `package` zgadza się z katalogiem, w którym leży plik,
- uruchomienie bez opcji `-p` daje kod wyjścia 2, niepoprawne dane (na przykład `-p 120`) kod 1, a poprawne kod 0.

Przykładowe rozwiązania

> **ZAJRZYJ TU, GDY UTKNIESZ**
>
> Najpierw spróbuj rozwiązać zadania samodzielnie. Jeśli utkniesz, przeczytaj wyjaśnienie, zamknij je i spróbuj napisać kod jeszcze raz. Przepisanie gotowego rozwiązania niczego nie uczy. Pokazany kod jest przykładowy: każdy program, który spełnia wymagania i daje ten sam wynik, jest poprawny.

Zadanie 1. Słaba referencja pod lupą

Kod

SlabaReferencja.java Java

```java
import java.lang.ref.WeakReference;

public class SlabaReferencja {
    public static void main(String[] args) {
        String raport = new String("Raport z laboratorium");
        WeakReference<String> slaba = new WeakReference<>(raport);

        System.gc();
        System.out.println("1. Silna referencja istnieje: " + slaba.get());

        raport = null; // usuwamy jedyną silną referencję
        System.gc();
        System.out.println("2. Po usunięciu silnej referencji: "
                + slaba.get());
    }
}
```

Terminal Terminal

```
> javac SlabaReferencja.java
> java SlabaReferencja
1. Silna referencja istnieje: Raport z laboratorium
2. Po usunięciu silnej referencji: null
> java -XX:+DisableExplicitGC SlabaReferencja
1. Silna referencja istnieje: Raport z laboratorium
2. Po usunięciu silnej referencji: Raport z laboratorium
```

Program uruchomiliśmy na JDK 25 z domyślnym kolektorem G1. Taki sam wynik pierwszego uruchomienia otrzymaliśmy także z kolektorami Serial, Parallel i ZGC.

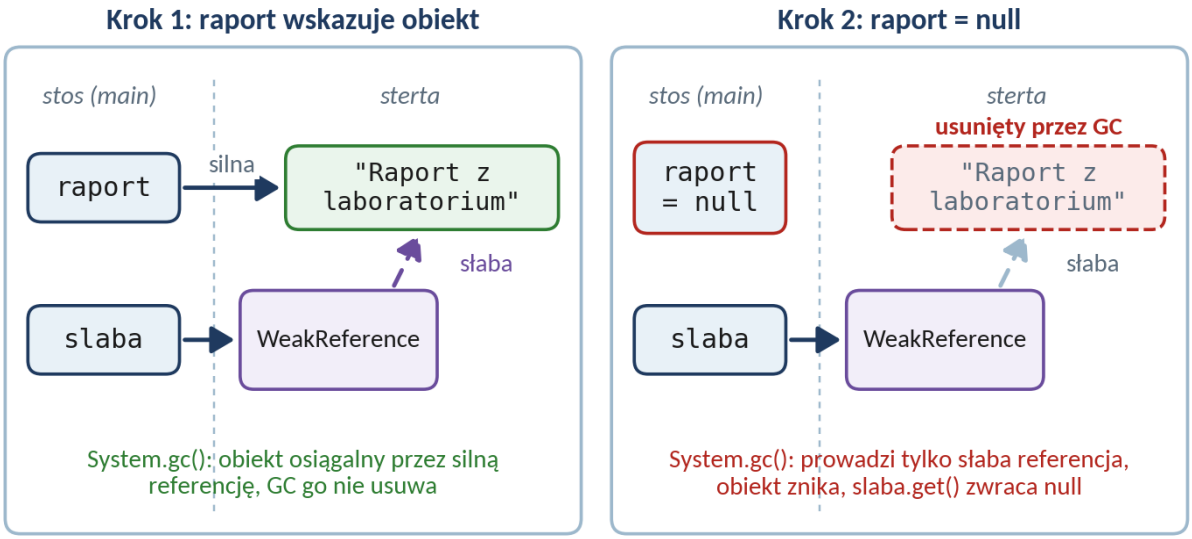
Wyjaśnienie kodu

Polecenie `new String(...)` tworzy na stercie nowy obiekt napisu. Nie jest to literał z puli napisów, więc obiekt może zostać usunięty, gdy przestanie być osiągalny. Zmienna `raport` jest do niego zwykłą, silną referencją. Obiekt klasy `WeakReference` przechowuje drugą, słabą referencję do tego samego napisu, a metoda `get()` zwraca napis albo `null`, jeśli GC już go usunął.

Przy pierwszym wywołaniu `System.gc()` do napisu prowadzi silna referencja ze zmiennej `raport`, która jest korzeniem (zmienną lokalną na stosie metody `main`). GC nie może więc usunąć obiektu, niezależnie od tego, czy prowadzą do niego także słabe referencje. Pierwszy wiersz zawsze pokazuje napis.

Po przypisaniu `raport = null` do napisu prowadzi już tylko słaba referencja. Taki obiekt GC usuwa przy najbliższym odśmiecaniu, które go obejmie. Przy domyślnym kolektorze G1 wywołanie `System.gc()` wykonuje

pełne odśmiecanie całej sterty, więc `slaba.get()` zwraca `null`. Przebieg obu kroków pokazuje poniższy schemat.



Rysunek 1. *Obiekt przed i po usunięciu silnej referencji*

Drugie uruchomienie pokazuje, że `System.gc()` jest tylko prośbą. Opcja `-XX:+DisableExplicitGC` każe JVM ignorować takie prośby, więc w naszym krótkim programie odśmiecanie nie nastąpi wcale, a obiekt przetrwa, mimo że prowadzi do niego tylko słaba referencja. Nie znaczy to, że słaba referencja go chroni: obiekt zostałby usunięty przy pierwszym odśmiecaniu uruchomionym przez samą JVM, na przykład gdy zabraknie miejsca w młodej generacji.

Tabela 3. Wynik slaba.get() w obu uruchomieniach

Moment	zwykłe uruchomienie	z opcją DisableExplicitGC
krok 1, silna referencja istnieje	napis	napis
krok 2, tylko słaba referencja	`null`	napis

Zadanie 2. Kalkulator ocen w wierszu poleceń

Kod

KalkulatorOcen.java Java

```java
package pl.psk.oceny;

import java.util.concurrent.Callable;
import picocli.CommandLine;

@CommandLine.Command(name = "oceny", mixinStandardHelpOptions = true,
        version = "oceny 1.0",
        description = "Zamienia liczbę punktów na ocenę")
public class KalkulatorOcen implements Callable<Integer> {

    @CommandLine.Option(names = {"-p", "--punkty"}, required = true,
            description = "Liczba zdobytych punktów")
    private int punkty;

    @CommandLine.Option(names = {"-m", "--maks"},
            description = "Maksymalna liczba punktów (domyślnie 100)")
    private int maks = 100;
```

```
    @CommandLine.Option(names = {"-i", "--imie"},
            description = "Imię studenta (domyślnie Student)")
    private String imie = "Student";

    static double ocena(double procent) {
        if (procent < 50) {
            return 2.0;
        } else if (procent < 60) {
            return 3.0;
        } else if (procent < 70) {
            return 3.5;
        } else if (procent < 80) {
            return 4.0;
        } else if (procent < 90) {
            return 4.5;
        } else {
            return 5.0;
        }
    }

    @Override
    public Integer call() {
        if (maks <= 0) {
            System.err.println("Maksymalna liczba punktów musi być dodatnia");
            return 1;
        }
        if (punkty < 0 || punkty > maks) {
            System.err.println("Liczba punktów musi być z zakresu od 0 do "
                    + maks);
            return 1;
        }
        double procent = 100.0 * punkty / maks;
        System.out.printf("%s: %d z %d pkt (%.1f procent)%n",
                imie, punkty, maks, procent);
        System.out.printf("Ocena: %.1f%n", ocena(procent));
        return 0;
    }

    public static void main(String[] args) {
        int exitCode = new CommandLine(new KalkulatorOcen()).execute(args);
        System.exit(exitCode);
    }
}
```

Plik leży w katalogu `src/main/java/pl/psk/oceny`. Projekt budujemy poleceniem `mvn package`, a potem uruchamiamy plik JAR z bibliotekami. Poniżej przebieg w terminalu systemu Linux (kod wyjścia pokazuje polecenie `echo $?`):

Terminal Terminal

```
> java -jar target/oceny-1.0-jar-with-dependencies.jar -p 78
Student: 78 z 100 pkt (78,0 procent)
Ocena: 4,0
> echo $?
0
> java -jar target/oceny-1.0-jar-with-dependencies.jar -p 45 --maks 50 -i Anna
Anna: 45 z 50 pkt (90,0 procent)
Ocena: 5,0
> java -jar target/oceny-1.0-jar-with-dependencies.jar -p 120
Liczba punktów musi być z zakresu od 0 do 100
> echo $?
1
> java -jar target/oceny-1.0-jar-with-dependencies.jar
Missing required option: '--punkty=<punkty>'
Usage: oceny [-hV] [-i=<imie>] [-m=<maks>] -p=<punkty>
Zamienia liczbę punktów na ocenę
  -h, --help              Show this help message and exit.
  -i, --imie=<imie>       Imię studenta (domyślnie Student)
  -m, --maks=<maks>       Maksymalna liczba punktów (domyślnie 100)
  -p, --punkty=<punkty>   Liczba zdobytych punktów
  -V, --version           Print version information and exit.
> echo $?
2
> java -jar target/oceny-1.0-jar-with-dependencies.jar -V
oceny 1.0
```

Wyjaśnienie kodu

Pierwszy wiersz pliku, `package pl.psk.oceny;`, umieszcza klasę w pakiecie. Pełna nazwa klasy to więc `pl.psk.oceny.KalkulatorOcen` i właśnie tę nazwę podaje element `mainClass` w pliku `pom.xml`. Katalogi `pl/psk/oceny` w `src/main/java` muszą odpowiadać nazwie pakietu. Klasy `Callable` i `CommandLine` pochodzą z innych pakietów, więc je importujemy. `Callable` leży w pakiecie `java.util.concurrent`, który nie jest importowany automatycznie.

Konfiguracja polecenia jest zapisana w adnotacjach. Picocli odczytuje je w czasie działania programu i na ich podstawie analizuje argumenty.

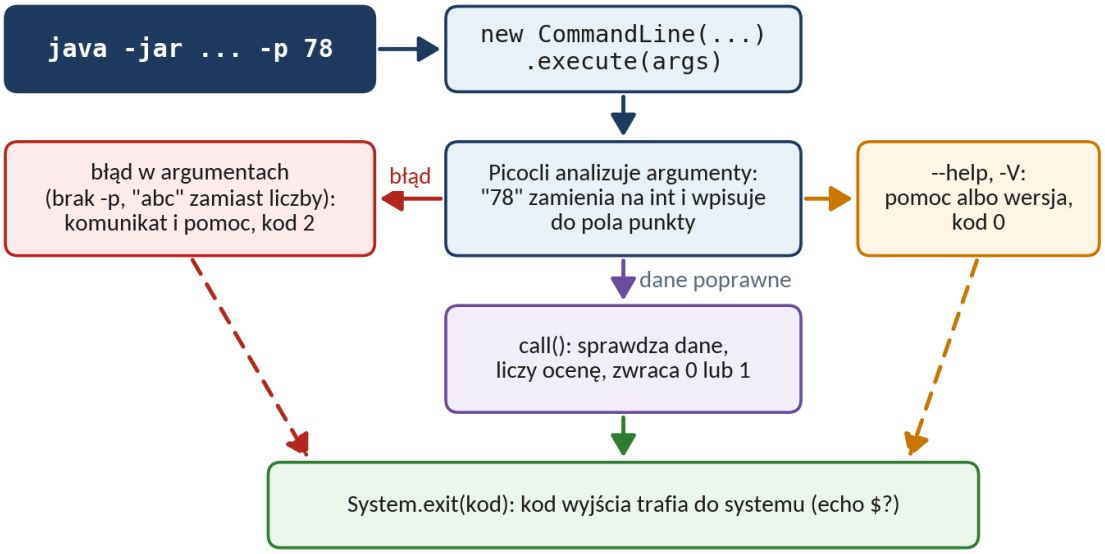
Tabela 4. Adnotacje użyte w rozwiązaniu

Adnotacja	Co robi w naszym programie
`@CommandLine.Command`	opisuje polecenie: nazwę `oceny` widoczną w pomocy, opis, wersję i standardowe opcje `-h` i `-V` (`mixinStandardHelpOptions = true`)
`@CommandLine.Option`	łączy opcje z wiersza poleceń z polami klasy; `required = true` czyni opcję `-p` obowiązkową
`@Override`	adnotacja wbudowana: kompilator sprawdza, że `call()` rzeczywiście implementuje metodę interfejsu `Callable`

Pola `maks` i `imie` mają wartości przypisane w deklaracji. Jeśli użytkownik nie poda odpowiedniej opcji, Picocli nie zmienia pola i zostaje wartość domyślna. Pole `punkty` jest typu `int`, więc Picocli sam zamienia tekst `"78"` na liczbę. Gdy zamiana się nie uda (na przykład dla `abc`), Picocli wypisuje komunikat *Invalid value for option '--punkty': 'abc' is not an int* oraz pomoc i nie wywołuje metody `call()`.

Metoda `execute()` przeprowadza cały proces pokazany na poniższym schemacie. Jeśli argumenty są poprawne, wywołuje `call()` i zwraca jej wynik. Przy błędzie w argumentach zwraca 2, a po wyświetleniu

pomocy lub wersji 0. W metodzie `main` przekazujemy ten kod do `System.exit()`, dzięki czemu trafia on do systemu operacyjnego i mogą z niego korzystać na przykład skrypty.



Rysunek 2. *Przebieg programu z Picocli i kody wyjścia*

Metoda `call()` najpierw sprawdza dane, których Picocli nie może ocenić samodzielnie: czy maksimum jest dodatnie i czy liczba punktów mieści się w zakresie. Komunikaty o błędach wypisuje na `System.err`, a nie na `System.out`, żeby nie mieszały się ze zwykłym wynikiem, gdy ktoś przekieruje wyjście programu do pliku. Wynik procentowy liczymy wyrażeniem `100.0 * punkty / maks`. Literał `100.0` jest typu `double`, więc całe działanie jest zmiennoprzecinkowe i dla 45 punktów na 50 daje 90,0, a nie wynik dzielenia całkowitego.

Metoda `ocena` sprawdza progi rosnąco. Każdy kolejny warunek `else if` zostaje sprawdzony tylko wtedy, gdy poprzednie były fałszywe, więc wystarczy podać górną granicę przedziału. Ocenę wypisujemy specyfikatorem `%.1f`, dlatego ocena 4 wyświetla się jako `4,0`.